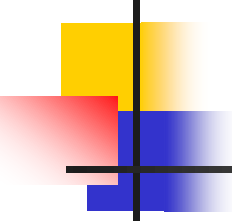


Introduction to Database Systems



Example of a Relation

Diagram illustrating the structure of a relation (table) named **STUDENT**.

The relation is defined by its **Attributes** and its **Tuples**.

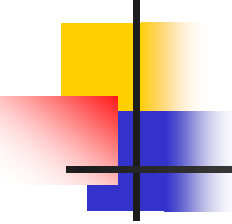
Attributes: Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa.

Tuples: Benjamin Bayer, Chung-cha Kim, Dick Davidson, Rohan Panchal, Barbara Benson.

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

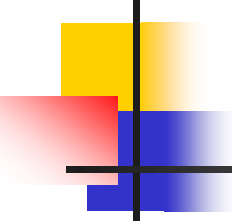
Figure 5.1

The attributes and tuples of a relation STUDENT.



Schemas versus Instances

- **Database Schema:** The *description* of a database. Includes descriptions of the database structure and the constraints that should hold on the database.
- **Schema Diagram:** A diagrammatic display of (some aspects of) a database schema.
- **Schema Construct:** A component of the schema or an object within the schema, e.g., STUDENT, COURSE.



University Database

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310



Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

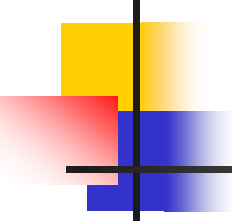
Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

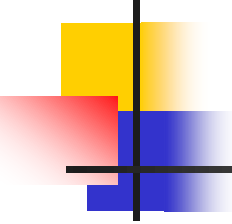
Figure 2.1

Schema diagram for the database in Figure 1.2.



Database schema VS Database state

- **Database State:** Refers to the content of a database at a moment in time.
- **Initial Database State:** Refers to the database when it is loaded
- **Valid State:** A state that satisfies the structure and constraints of the database.
- The **database schema** changes *very infrequently*. The **database state** changes *every time the database is updated*.
- **Schema** is also called **intension**, whereas **state** is called **extension**.



Three-Schema Architecture

- Three of four important characteristics of the database approach:
 1. Self-describing of a DB (Schema)
 2. Insulation between programs and data
 3. Support of multiple views of the data
- **Three-Schema Architecture** : it was proposed to help achieve and visualize these characteristics

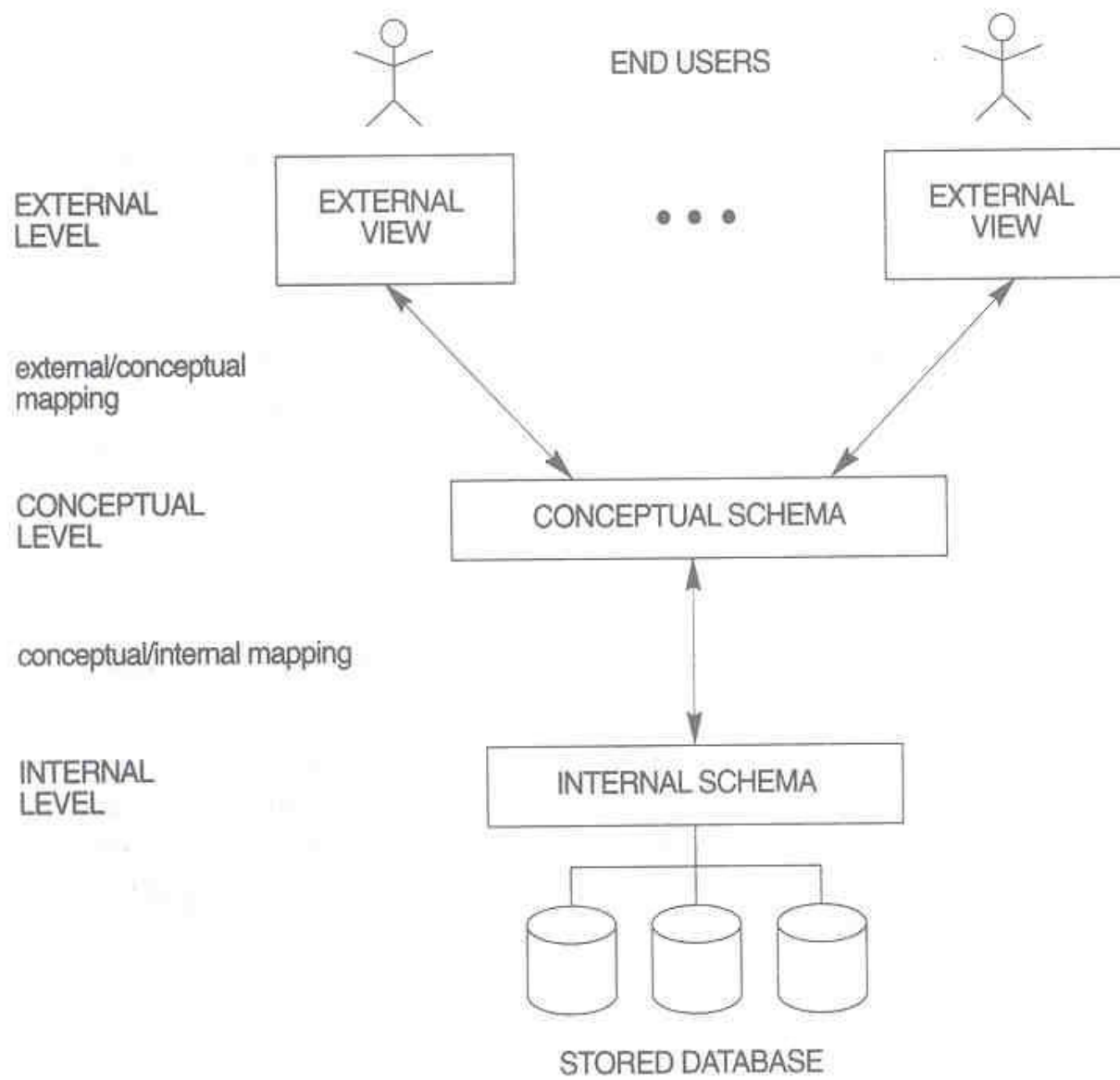
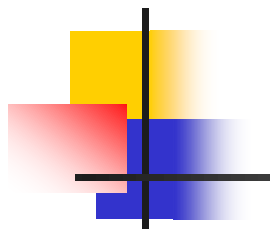
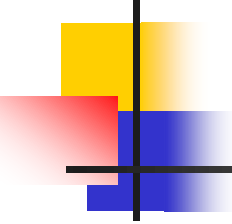
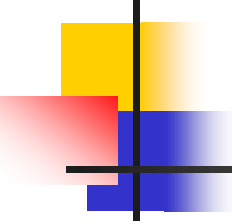


Figure 2.2 Illustrating the three-schema architecture.



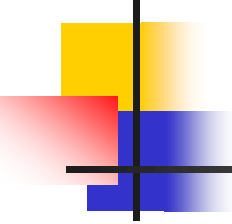
Three-Schema Architecture

- Defines DBMS schemas at *three levels*:
 - **Internal schema** at the internal level to describe physical storage structures and access paths. Typically uses a *physical* data model.
 - **Conceptual schema** at the conceptual level to describe the structure and constraints for the *whole* database for a community of users. Uses a *conceptual* or an *implementation* data model.
 - **External schemas** at the external level to describe the various user views. Usually uses the same data model as the conceptual level.



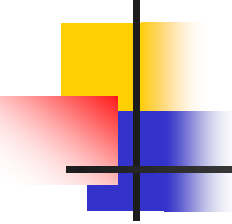
Three-Schema Architecture

- **Mappings** among schema levels are needed to transform requests and data. Programs refer to an external schema and are mapped by the DBMS to the internal schema for execution.



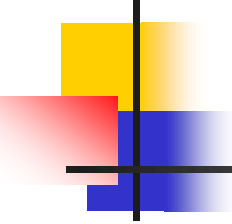
Data Independence

- **Logical Data Independence:** The capacity to change the conceptual schema without having to change the external schemas and their application programs.
- **Physical Data Independence:** The capacity to change the internal schema without having to change the conceptual schema.



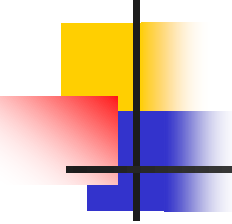
DBMS Languages

- **Data Definition Language (DDL)**: Used by the DBA and database designers to specify the *conceptual schema* of a database. In many DBMSs, the DDL is also used to define internal and external schemas (views). In some DBMSs, separate **storage definition language (SDL)** and **view definition language (VDL)** are used to define internal and external schemas.



DBMS Languages

- **Data Manipulation Language (DML):**
Used to specify database retrievals and updates.
- DML commands (**data sublanguage**) can be *embedded* in a general-purpose programming language (**host language**), such as COBOL, C or an Assembly Language.
- Alternatively, *stand-alone* DML commands can be applied directly (**query language**).



DBMS Languages

- **High Level or Non-procedural Languages:** e.g., SQL, are *set-oriented* and specify what data to retrieve than how to retrieve. Also called *declarative* languages.
- **Low Level or Procedural Languages:** record-at-a-time; they specify *how* to retrieve data and include constructs such as looping.

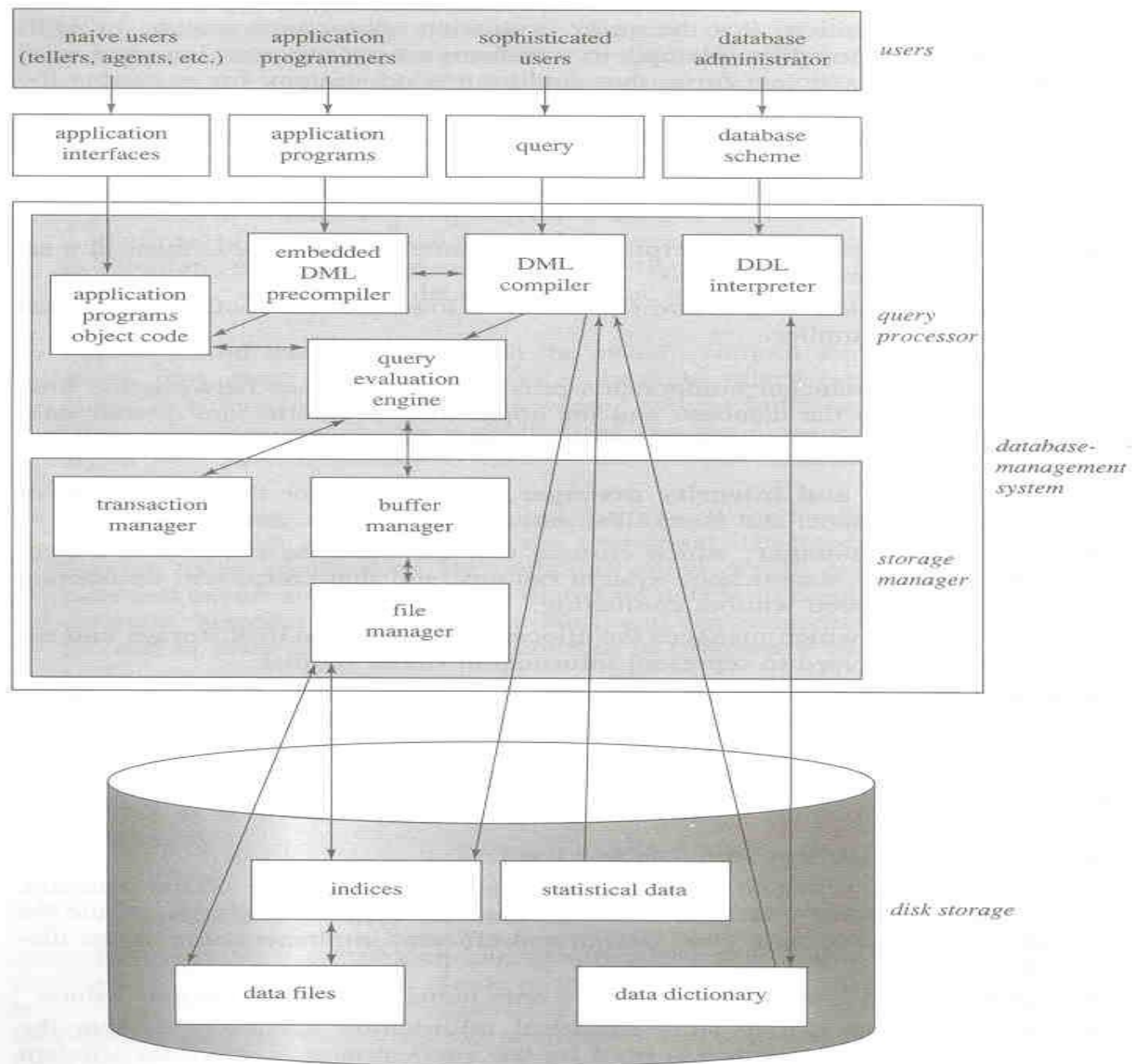
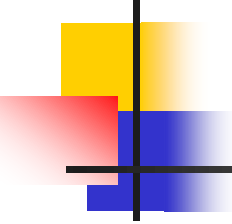
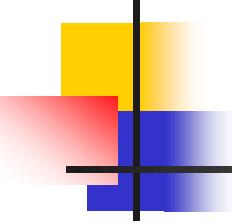


Figure 1.6 System structure.



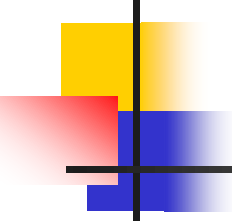
System Structure (Query processor)

- **DML-compiler**, which translates DML statements in a query language into low-level instructions that the query evaluation engine understands. In addition, the DML compiler attempts to transform a user's request into an equivalent but more efficient form, thus finding a good strategy for executing the query.



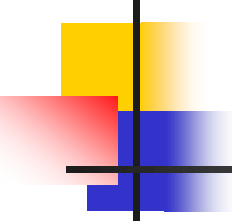
System Structure (Query processor)

- **Embedded DML precompiler**, which converts DML statements embedded in an application program to normal procedure calls in the host language.
- **DDL interpreter**, which interprets DDL statements and records them in a set of tables containing metadata.



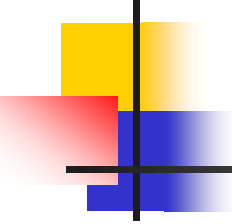
System Structure (Query processor)

- **Query evaluation engine**, which executes low-level instructions generated by the DML compiler.



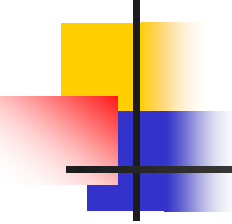
System Structure (Storage mgr)

- **Authorization and integrity manager**, which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
- **Transaction manager**, which ensures that the database remains in a consistent (correct) state despite system failures, and that concurrent transaction executions proceed without conflicting.



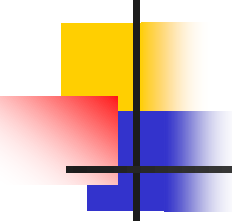
System Structure (Storage mgr)

- **File manager**, which manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer manager**, which is responsible for fetching data from disk storage into main memory and deciding what data to cache in memory.



System Structure (physical system)

- **Data files**, which store the database itself.
- **Data dictionary**, which stores metadata about the structure of the database. The data dictionary is used heavily. Therefore, great emphasis should be placed on developing a good design and efficient implementation of the dictionary.



System Structure (physical system)

- **Indices**, which provide fast access to data items that hold particular values.
- **Statistical data**, which store statistical information about the data in the database.